数字校准模块说明文档

吴林晓

总览

文件夹内包含：
1. 行为级模型(matlab, 版本 R2023a)
2. FGPA 工程(vivado, 版本 2018.3)
3. 电路文件(保存在服务器上，未提取出来)
4. 文档(包含组会 ppt，毕业论文，FPGA 说明文档，算法解释文档)
5. 相关文献

按照研究内容分类：
1. 针对 16bits SAR 的 基于 dither 注入的 LMS 校准算法(行为级+FPGA+电路级)
2. 针对 16bits SAR 的 低位校准高位校准算法(行为级+FPGA)
3. 针对 18bits SAR 的 基于 dither 注入的 LMS 校准算法(行为级+FPGA+电路级)
4. 针对 18bits SAR 的 低位校准高位校准算法(行为级+FPGA)

需要注意，FPGA 工程的路径不能存在中文

以研究内容顺序进行介绍

目录

总览.....1
1. 16bits SAR ADC，基于 dither 注入的 LMS 校准算法.....3
1.1. LMS 校准原理说明.....3
1.2. 行为级模型.....3
1.3. FPGA 工程.....6
1.4. 电路修改部分与联合仿真操作.....11
2. 16bits SAR ADC，低位校准高位.....14
2.1. 行为级模型.....14
2.1.1. 原始文件：.....14
2.1.2. 校准数字码产生文件.....14
2.2. FPGA 工程与测试方式.....15
2.2.1. 工程架构.....15
2.2.2. 测试流程.....16
3. 18bits，基于 dither 注入的 LMS 校准算法.....19
3.1. 行为级模型.....19
3.1.1. 原文件.....19
3.1.2. 其他程序与行为级测试结果.....21
3.2. FPGA 工程与测试方式.....23
3.2.1. 工程架构：.....23
3.2.2. LMS 校准.....24
3.2.3. FPGA 工程测试.....25
3.3. 电路修改.....29
4. 18bits，低位校准高位.....31
4.1. 行为级模型.....31
4.1.1. 原文件：.....31
4.1.2. 数字码产生文件.....31
4.2. FPGA 工程与测试方式.....31

1.16bits SAR ADC，基于 dither 注入的 LMS 校准算法

1.1. LMS 校准原理说明

见毕业论文章节 3.3

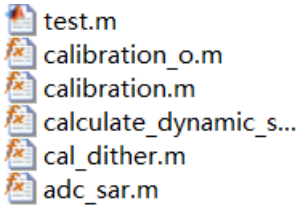
1.2. 行为级模型

1.F:\16bits\dither-based LMS calibration\原 dither 校准行为级代码

此为最初的 dither 校准 matlab 代码与介绍 ppt（林梦林），具体介绍见文件夹内 txt，其 ADC 模型不是最终使用的 16bits ADC 模型

2.16bits 的 dither 校准模块：

F:\16bits\dither-based LMS calibration\16bits dither 行为级仿真\16bit_dither 校准



图表 1-1 MATLAB 模块

Adc_sar：16bits SAR ADC 的行为级模型（与电路文件对应）

Cal_dither：LMS 校准模块，传入数字码，传出校准后的权重

Calcualte_dynamic_spec：FFT 模块，传入 adout 进行 FFT

Calibration：低位校准高位的校准算法(此处不调用，在与 LMS 校准比较校准效果时可以调用)

Test:主程序，运行顺序如下：

1. 校准前进行一次 ADC 转换，用 128 个点进行 FFT

```
%校准前转换
L_real= log2((WL_dummy+sum(WLda) + W_Cc)/W_Cc);
Wda_real = Wda.*2^L_real;%高段因为低端的dummy加入，真实权重发生改变
Wdit_real=Wdit*2^L_real;
Wall(1:M+L+1) = [Wda_real WLda,1/2]; %所有位真实权重
weight =Wall(1:M+L+1);
dout=zeros(1,18);
for i = 1:d_len
    ctrl=0;
    Vip = 0.49*Vref*sin(i*delta_ph)+Vcm; % defining input signal 输入单频信号，幅度近似满量程，用s
    Vin = -0.49*Vref*sin(i*delta_ph)+Vcm;
    dout = adc_sar(Vip,Vin,Vref,Vcm,M,L,Cm_p,Cm_n,Cl_p,Cl_n,Cd1_p,Cd1_n,Cd2_p,Cd2_n,Ca_p,Ca_n,Cp1
    dout=2*dout-1;
    adout(i) = dout * weight';
end

[pow,SNR,SNDR,ENOB,SFDR,THD,HD] = calculate_dynamic_spec(adout,OSR);
```

图表 1-2 校准前

2. 进行校准，生成校准信号后导入 Cal_dither，Cal_dither 内部调用 Adc_sar 进行 dither 注入和转换，经过多个样本点的迭代(最终选定用 65536 个样本点进行迭代)，生成最终的权重

```
u=zeros(1,18);

%uk = 5e-6;%校准因子
uk =2^(-10);%校准因子

u = weight*uk;
u = [uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk];
%u(15:18)=u(15:18)*8;
%u(10:14)=u(10:14)*4;
%u(5:9)=u(5:9)*2;%构成收敛系数矩阵
ud=u(7);
%%%%%%%%%%%%%%%%%%%%%%%%%%
for i=1:ns
Vip_cal(i,1) = 0.45*Vref*sin(0.1*i*delta_ph)+Vcm;% defining input signal 输入单频信号，用sar adc对其进行转换，并将转
Vin_cal(i,1) = -0.45*Vref*sin(0.1*i*delta_ph)+Vcm;
% Vip_cal(i,1) = 0.45*Vref*sin(2*pi*1/500*i)+Vcm;% defining input signal 输入单频信号，用sar adc对其进行转换，并将转
% Vin_cal(i,1) = -0.45*Vref*sin(2*pi*1/500*i)+Vcm;
end
```

图表 1-3 进行校准

3. 校准后进行一次 ADC 转换，用校准后的权重与得到数字码加权相加，用 128 个点进行 FFT

```
%校准后转换
for i = 1:d_len
    ctrl=0;
    Vip = 0.49*Vref*sin(i*delta_ph)+Vcm; % defining input signal 输入单频信号，幅度近似满量程，用sar adc对其进行转换
    Vin = -0.49*Vref*sin(i*delta_ph)+Vcm;
    dout = adc_sar(Vip,Vin,Vref,Vcm,M,L,Cm_p,Cm_n,Cl_p,Cl_n,Cd1_p,Cd1_n,Cd2_p,Cd2_n,Ca_p,Ca_n,Cp1_p,Cp1_n,Cp2_p,C
    dout=2*dout-1;
    adout(i) = dout * weight_final';
end
[pow,SNR,SNDR,ENOB,SFDR,THD,HD] = calculate_dynamic_spec(adout,OSR);
```

图表 1-4 校准后测试

参数设置：

1. 非理想因素：

```
Cu = 30e-15;     %单位电容值
del_Cu = Cu*0.03;% 定义工作电容的设计值和分布的均方根值
alfa =0.02; beta =0.02;  % 定义 alfa底板和beta顶板到地的归一化寄生电容。
% 此值是估值，实际取值应从版图提取。底板寄生相对没那么重要，顶板寄生较重要，大概是0.1。正确科学的方法是从版图提取，但
del_Compos = 10e-6;        % 定义比较器失调的均方根值。均方根值=标准差 σ
del_Compvn = 47e-6;      % 定义比较器噪声
pkt = 1;                   % 是否加入kT/C噪声

%*****************以下是理想情况***************
 del_Cu = 0;
 alfa =0; beta =0;
 del_Compos = 0;     %定义比较器失调的均方根值
 del_Compvn = 0;   %定义比较器噪声
 pkt = 0;          %是否加入kT/C噪声
```

图表 1-5 非理想因素设置

一般默认设置电容失配为 3%，alfa beta=0.02，热噪声由 del_Compos, del_Compvn 和 pvt 控制，比较器噪声通常取与 LSB 相同的数量级

2. 校准参数：

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
ns=65536; %校准sample数
weight_cal=zeros(ns,18); %存储校准位数的矩阵

u=zeros(1,18);

uk =2^(-10);%校准因子
u = [uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk];

% u=2^(-17);
% u = weight*uk;
%u(15:18)=u(15:18)*8;
%u(10:14)=u(10:14)*4;
%u(5:9)=u(5:9)*2;%构成收敛系数矩阵
ud=u(7);
```

图表 1-6 校准参数设置

Ns 即校准的样本点，此处对权重进行 65536 次迭代。

对于收敛因子，有两种取法：

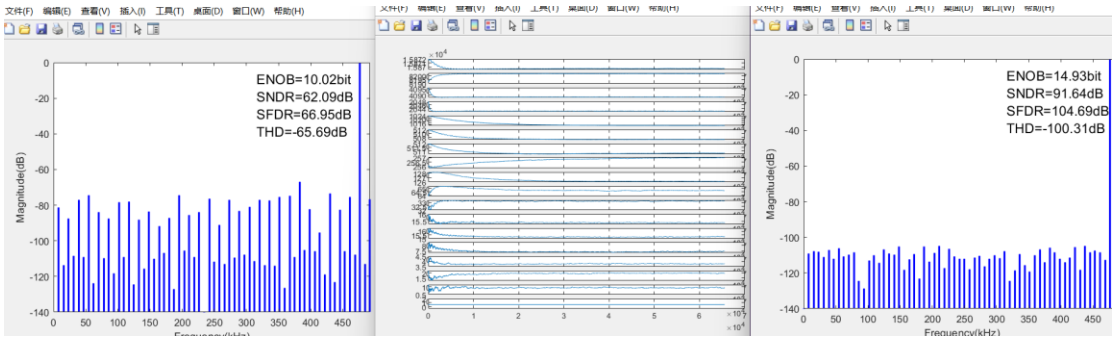
若选取所有权重的收敛因子大小相同，uk 取 2^(-10)，这种取法的优点是硬件开销更加友好，在 verilog 中可以用移位代替乘法；

若选取不同权重的收敛因子大小不同，即 u＝weight*uk，则按照注释处设置，在林梦林的 ppt 中，这种取法会更快的收敛，但实际上两种取法在校准效果和收敛速度的区别不大，但前者的硬件开销更为友好。（参考文献中未看见取 u = weight*uk 这种做法）。

以毕业论文为节点，在此之前的工程都取 u = weight*uk(即 16bits 的校准工程)，18bits 的则选取所有权重的收敛因子大小相同，因此 FPGA 硬件开销显著更少。

运行方式：

设置好非理想因素后，运行 test 程序，一段时间后得到 3 张图：



图表 1-7 fft 结果

从左到右分别为：校准前 FFT，权重变化图，校准后 FFT。

3. 其他工程文件：

文件夹内其余工程都基于上述工程，改写了主程序结构由于仿真验证，可以不用关注：

16bit_dither 校准 - 曲线绘制：多次仿真，得到校准效果与样本量的关系曲线（learning curve），Loop 后缀表示多次仿真，最终导出数据求平均值进行绘图。

16bit_dither 校准 - 频率：研究校准效果与频率的关系

16bit_dither 校准 - 拼接：由于在前仿对 ADC 数字码进行拼接，以节约仿真时间，此处探究拼接是否会影响校准效果（结果是几乎不会，在大四下 ppt 中

有汇报）

16bit 校准方法比较：比较 dither 前台校准，后台校准和低位校准高位的校准效果

部分行为级测试结果见毕业论文:

章节 3.4 数字校准算法行为级建模

章节 4.4 模拟扰动注入对校准效果的影响

1.3. FPGA 工程

使用的 FPGA 型号和 IO 设置在文件夹 FPGA 参考文档中有介绍，并包含原厂的教学。

具体的工程架构已经在论文章节 **4.1.3.1** 基于扰动注入的数字校准模块介绍，此处的工程为数字前台校准模块，输入校准和测试使用的数字码，大小分别为 18*131072 和 18*128(列*行),最终输出 20 位的纯二进制数字码，在 ila 中直接转为十进制后导入 MATLAB 进行 FFT。具体的测试方法如下：

1. 得到数字码

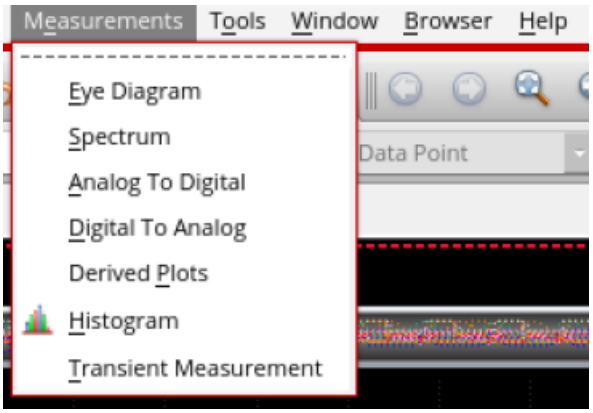
若使用 MATLAB 的 ADC 模型生成数字码：

使用工程 F:\16bits\dither-based LMS calibration\FPGA\ADC

与前述的行为级模型一致，设定好非理想因素后运行，得到校准结果和数字码

Dout_mem 为校准码，dout_test 为测试码

若使用电路前仿得到数字码，在仿真界面保存数字码输出后，在波形界面点击 measurements->analog to digital



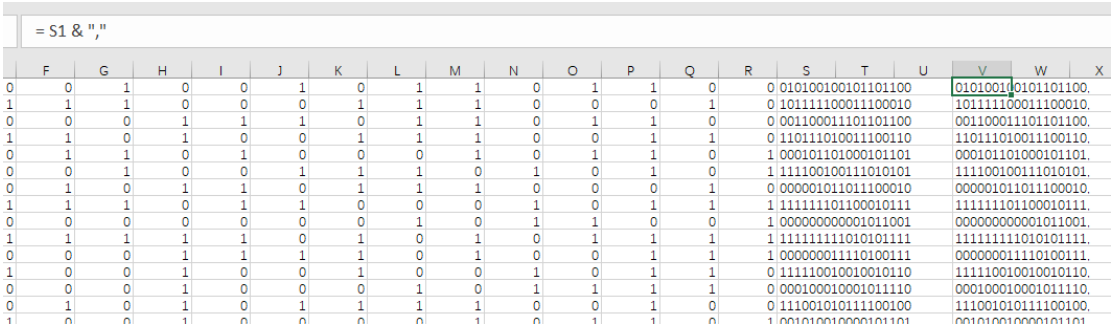
图表 1-8 前仿数据处理

设定 center 为 1.65 后输出，保存为 csv 格式，即得到数字码的 csv 文件。

图表 1-9 前仿数据处理

2. 导入数字码至 FPGA

文件夹中的 414.xlxs 为范例，最终转为 18 为数字码+逗号的格式



= S1 & ","

F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
0	0	1	0	0	1	0	1	1	0	1	1	0	0	010100100101101100		01010010101101100,		
1	1	1	0	0	0	1	1	1	0	0	0	1	0	101111100011100010		101111100011100010,		
0	0	0	1	1	1	0	1	1	0	1	1	0	0	001100011101101100		001100011101101100,		
1	1	0	1	0	0	1	1	1	0	0	1	1	0	110111010011100110		110111010011100110,		
0	1	1	0	1	0	0	0	1	0	1	1	0	1	000101101000101101		000101101000101101,		
0	0	1	0	0	1	1	1	0	1	0	1	0	1	111100100111010101		111100100111010101,		
0	1	0	1	1	0	1	1	1	0	0	0	1	0	000001011011100010		000001011011100010,		
1	1	1	0	1	1	0	0	0	1	0	1	1	1	111111101100010111		111111101100010111,		
0	0	0	0	0	0	0	1	0	1	1	0	0	1	000000000001011001		000000000001011001,		
1	1	1	1	1	0	1	0	1	0	1	1	1	1	111111111010101111		111111111010101111,		
0	0	0	1	1	1	1	0	1	0	0	1	1	1	000000011110100111		000000011110100111,		
1	0	0	1	0	0	1	0	0	1	0	1	1	0	111110010010010110		111110010010010110,		
0	0	0	1	0	0	0	1	0	1	1	1	1	0	000100010001011110		000100010001011110,		
0	1	0	1	0	1	1	1	1	0	0	1	0	0	111001010111100100		111001010111100100,		
1	0	0	1	0	0	0	0	1	0	1	1	0	1	001010010000101101		001010010000101101		

图表 1-10 csv 文件示例

新建 txt，按照语法输入数字码（示例在 coe_4.14 文件夹中），注意最后一行以；结束

```
memory_initialization_radix = 2;
memory_initialization_vector =
010100100101101100,
101111100011100010,
001100011101101100,
110111010011100110,
000101101000101101,
111100100111010101,
000001011011100010,
111111101100010111,
000000000001011001,
111111111010101111,
```

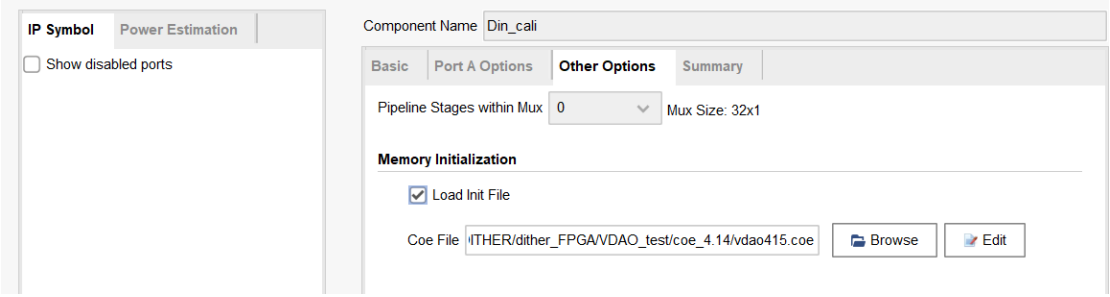
图表 1-11 数字码导入 coe 文件

修改文件后缀为 coe

test415.coe	2025/4/15 11:19	COE 文件	3 KB
vdao415.coe	2025/4/15 11:18	COE 文件	2,689 KB

图表 1-12 coe 文件

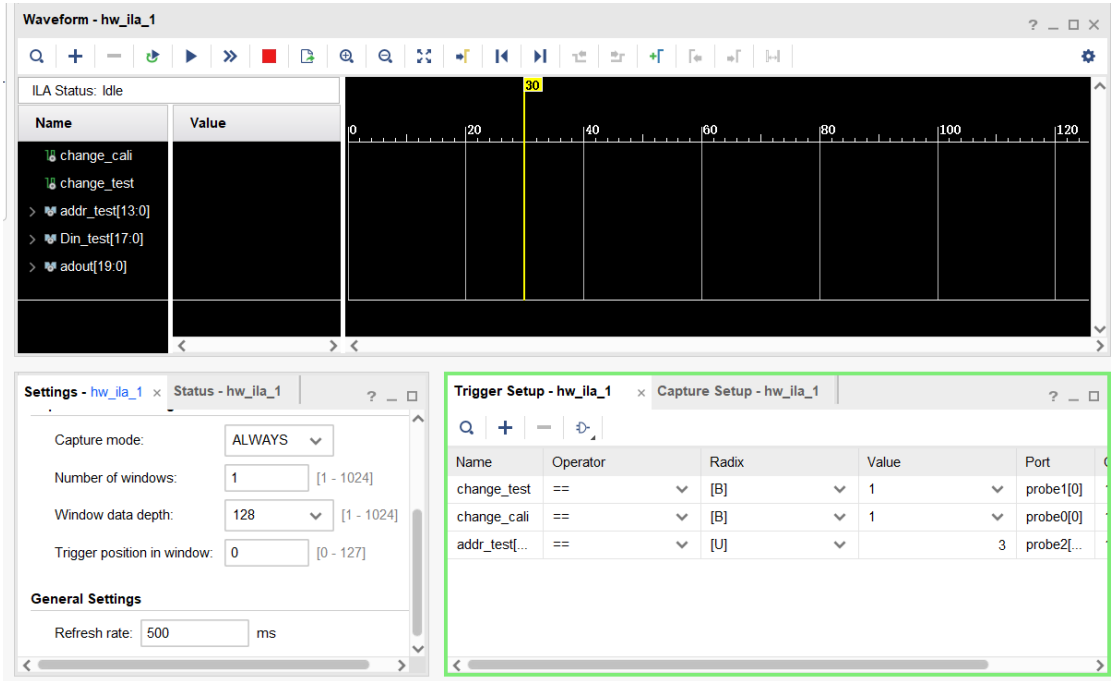
随后导入两个 rom ip 核中，即完成了数字码的导入



图表 1-13 coe 导入 rom

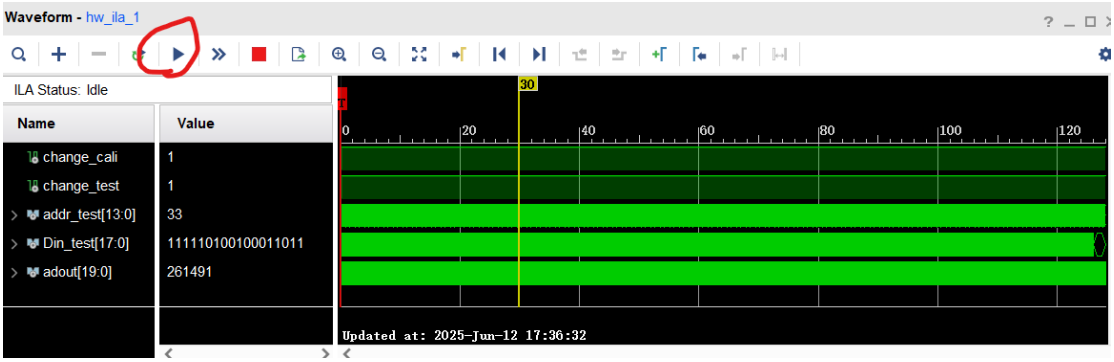
3. 运行工程与结果输出

将比特流写入 FPGA 后，自动弹出 ila 调试界面：



图表 1-14 ila 调试界面

上方为监控 FPGA 内部信号的窗口，下方为触发信号的设置，按照图示设置。



图表 1-15 ila 调试

点击运行后，开始检测触发信号，按照顺序分别按下 FPGA 的 PL LED2，PL LED3 按钮（分别是开始校准，开始重新编码的使能按钮）

图表 1-16 用户按钮

得到 adout 的输出，右键波形设置为十进制输出后，在 tcl 输入命令

write_hw_ila_data -csv_file E:/2024_adc/DITHER/FFT.csv [upload_hw_ila_data hw_ila_1]

此处设置了输出 csv 文件的路径与名字，得到的 csv 文件如下（FFT612）：

	A	B	C	D	E	F	G	H
1	Sample in	Sample in	TRIGGER	change_ca	change_te	addr_test[	Din_test[1	adout[19:
2	0	0	1	1	1	3	1.1E+15	86794
3	1	1	0	1	1	4	1.1E+17	192516
4	2	2	0	1	1	5	1.01E+14	52944
5	3	3	0	1	1	6	1.11E+17	223345
6	4	4	0	1	1	7	1.01E+12	25805
7	5	5	0	1	1	8	1.11E+17	246207
8	6	6	0	1	1	9	1011001	7714
9	7	7	0	1	1	10	1.11E+17	259136
10	8	8	0	1	1	11	1.11E+10	229
11	9	9	0	1	1	12	1.11E+17	261018
12	10	10	0	1	1	13	1E+14	3992
13	11	11	0	1	1	14	1.11E+17	251691
14	12	12	0	1	1	15	1.01E+15	18685
15	13	13	0	1	1	16	1.1E+17	231955
16	14	14	0	1	1	17	1E+16	43033
17	15	15	0	1	1	18	1.01E+17	203514
18	16	16	0	1	1	19	1.1E+16	74945
19	17	17	0	1	1	20	1.11E+16	168822
20	18	18	0	1	1	21	1E+17	111675
21	19	19	0	1	1	22	1.01E+16	130855
22	20	20	0	1	1	23	1.01E+17	150053
23	21	21	0	1	1	24	1.1E+15	92889
24	22	22	0	1	1	25	1.1E+17	186783
25	23	23	0	1	1	26	1.1E+14	58193
26	24	24	0	1	1	27	1.11E+17	218695
27	25	25	0	1	1	28	1.1E+12	29757
28	26	26	0	1	1	29	1.11E+17	243042
29	27	27	0	1	1	30	1.01E+08	10024
30	28	28	0	1	1	31	1.11E+17	257728

图表 1-17 fpga 数据输出

其中的 H 列为输出的 10 进制数，共 128 位。

4. 进行 FFT 测试

打开文件夹：“FFT - vdao 直接导入”，其中 test 为主程序

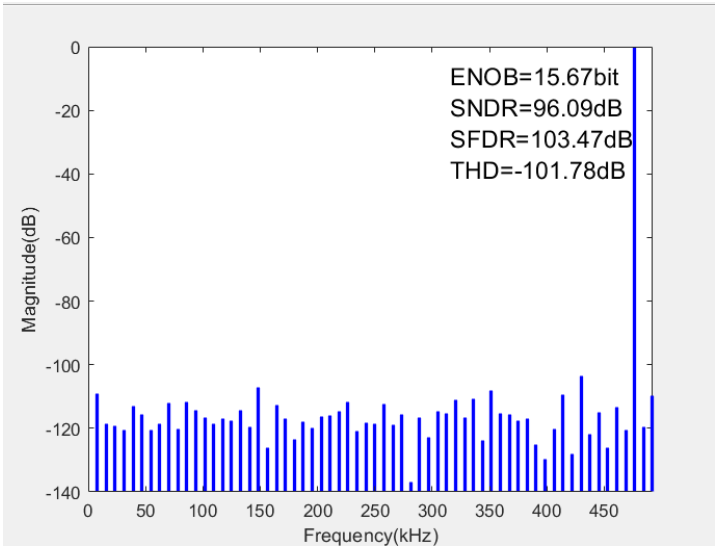
```
clear all;
% d_len=1024;
d_len=128;
OSR=1;
filename = 'E:/2024_adc/DITHER/FFT527.csv';

% 检查文件是否存在
if ~isfile(filename)
    error('File not found: %s', filename);
end
% 读取 H 列的 2~129 行数据
dataRange = 'H2:H129';
% dataRange = 'I2:I1029';
% 使用 readmatrix 读取数据
M = readmatrix(filename, 'Range', dataRange);

MAT=M;
```

图表 1-18 fft 测试

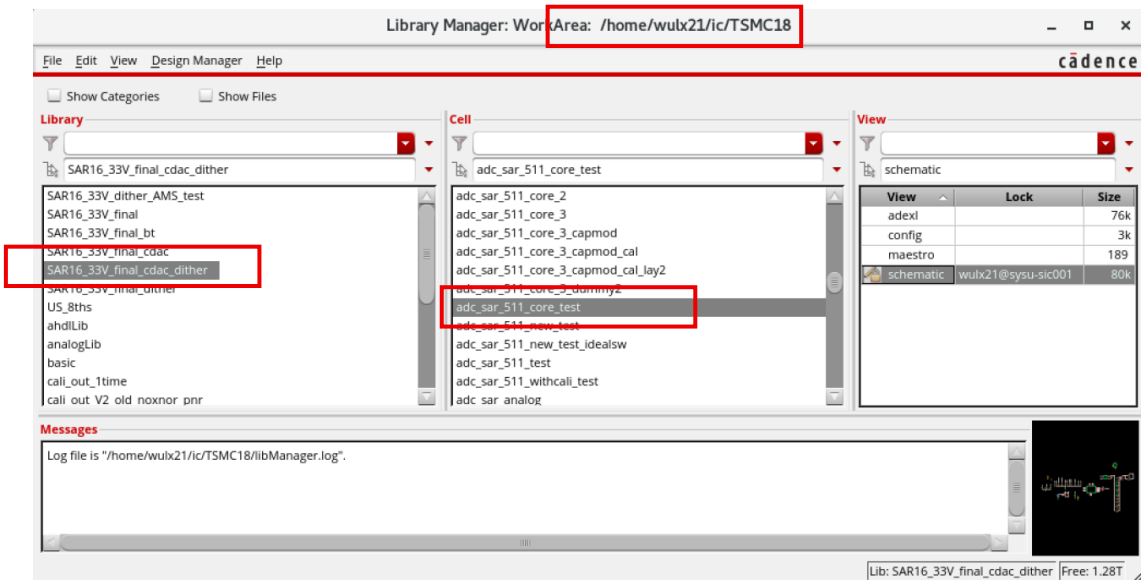
设置 csv 文件的路径和数据位置后，即读取上述的 128 位 10 进制数进行 FFT，得到频谱和参数：



图表 1-19 fft 结果

1.4. 电路修改部分与联合仿真操作

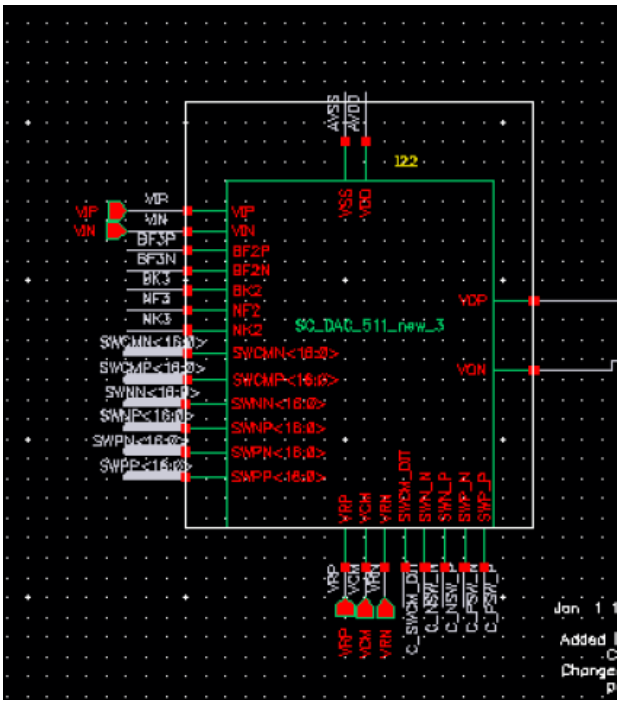
1.4.1. 电路位置



图表 1-20 电路位置

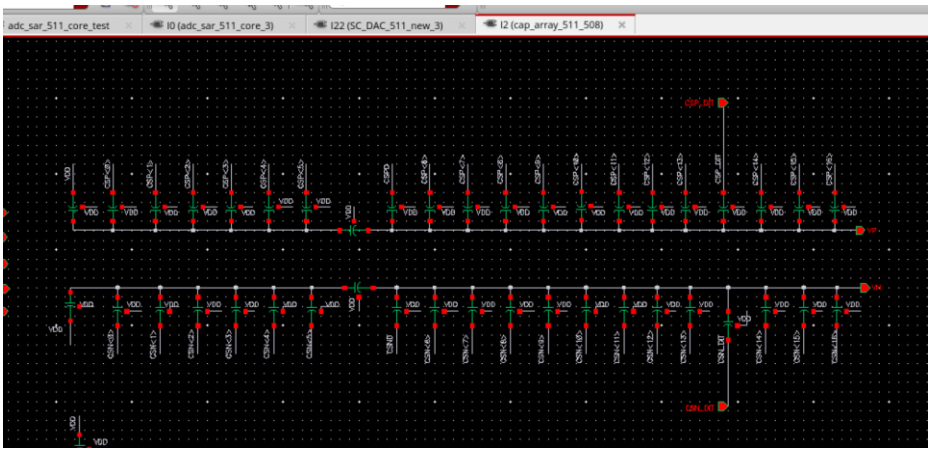
其余四个库不建议删除，内部的器件可能被调用过。但 dither 校准的最终测试基本上在图示的工艺库内

1.4.2. 电路修改内容



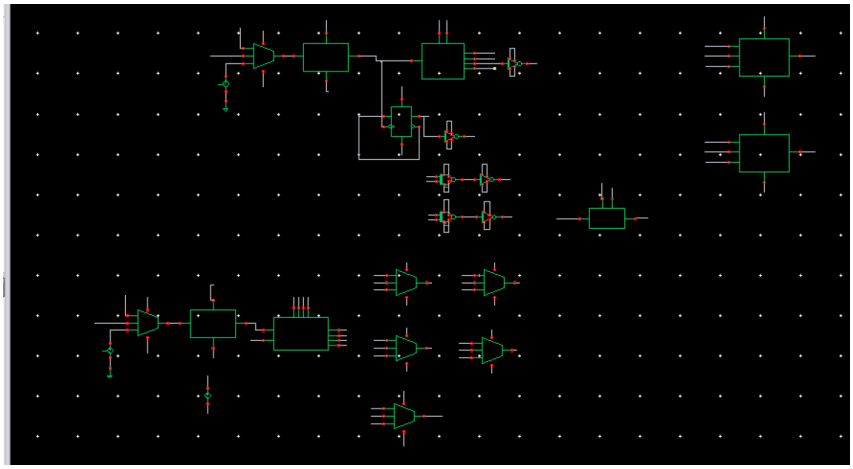
图表 1-21 CDAC

引出 5 个数字控制信号，控制 dither 电路时序，当前的时序是先负 dither 后正 dither。



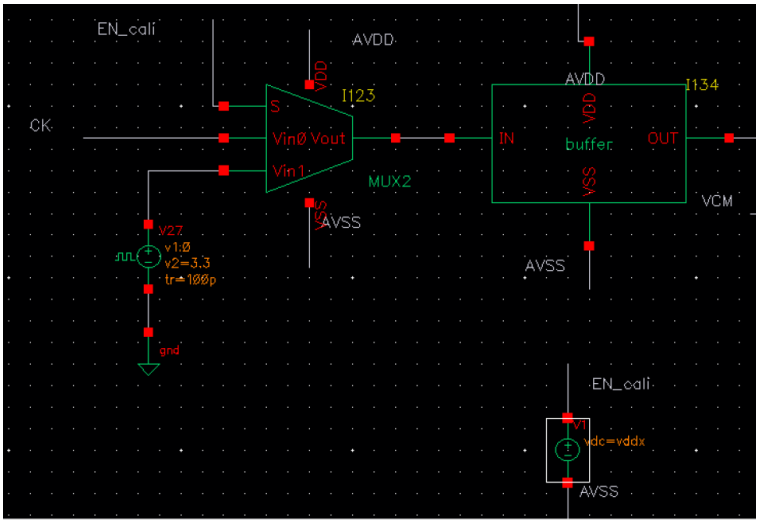
图表 1-22 CDAC dither 电容

CDAC 中 MSB 段引入两个 dither 电容。



图表 1-23 时序控制电路

一些时序控制电路，用于产生 dither 电容下极板的切换信号，以及校准阶段和正常转换阶段的切换电路。

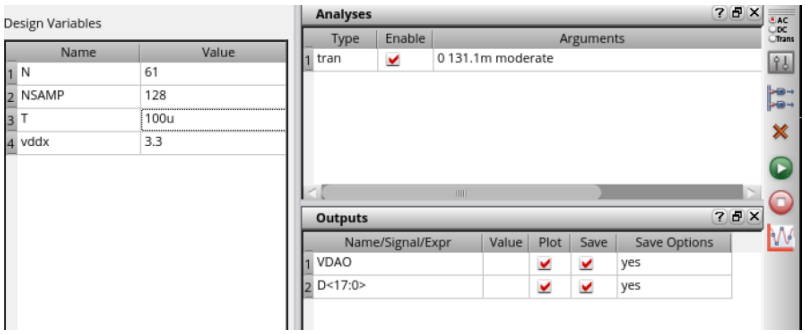


图表 1-24 校准使能信号

EN_cali 是校准的使能信号，设置为 vddx 时，ADC 工作在校准阶段，进行一次采用两次转换，分别插入负正 dither；设置为 0 时，ADC 工作在正常转换阶段。

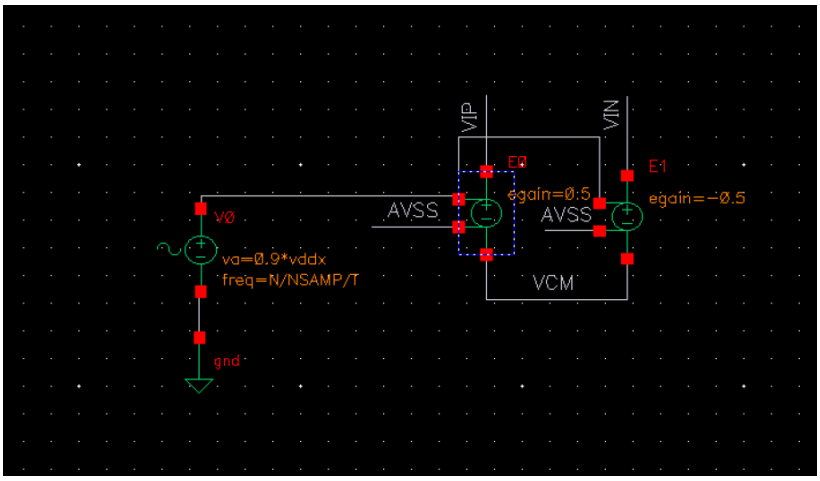
1.4.3. 仿真设置

打开上述电路，load state“dither_cali”



图表 1-25 仿真设置

经实测，校准时输入信号频率较低时，校准效果明显(或者说收敛速度快)，此处输入信号 0.8VDD, 4.765625kHz。



图表 1-26 输入信号

校准阶段，由于 dither 的注入，输入信号不能满摆幅，选取大小为 0.9*vddx。

完成设置后，运行仿真，每 2us 完成一次正负 dither 注入，即一个 sample，按照行为级模型。完成校准需要 65536 个 samples，但仿真时间过长。在实际中运行一晚上后得到 4000 多个 samples，直接进行拼接得到 65536 个 samples（单纯的复制粘贴，注意每两个 samples 是对相同采样信号的两次转换，因此注意复制的内容，在 vdao 上观察无误后复制）。

2.16bits SAR ADC，低位校准高位

2.1. 行为级模型

2.1.1. 原始文件

路径：F:\calibration\16bits\lower bits-based calibration\16bit 校准_过采样
由于原始的低位校准高位文件不是我写的，在此不赘述。

```
del_Cu = Cu*0.02e-2;        % 定义工作电容的设计值和分布的均方根值
alpha =0.2; beta =0.2;   % 定义 alfa底板和beta顶板到地的归一化寄生电容。此

del_Compos = 10e-6;        % 定义比较器失调的均方根值。均方根值=标准差 σ
del_Compvn = 47e-6;     % 定义比较器噪声
pkt = 1;                   % 是否加入kT/C噪声

%*****************以下是理想情况****************
% del_Cu = 0;        %定义工作电容的设计值和分布的均方根值
% alpha =0; beta =0;  %定 底板和顶板到地的归一化寄生电容
% del_Compos = 0;     %定义比较器失调的均方根值
% del_Compvn = 0;  %定义比较器噪声
% pkt = 0;         %是否加入kT/C噪声
```

图表 2-1 原文件非理想因素

需要注意，原文件的电容失配开的太小（0.02%）,难以观察校准效果，建议设置为 3%，如前文所述。

2.1.2. 校准数字码产生文件

为了方便 FPGA 工程的测试，在原文件的基础上修改了一版，将校准过程的数字码提取出来。

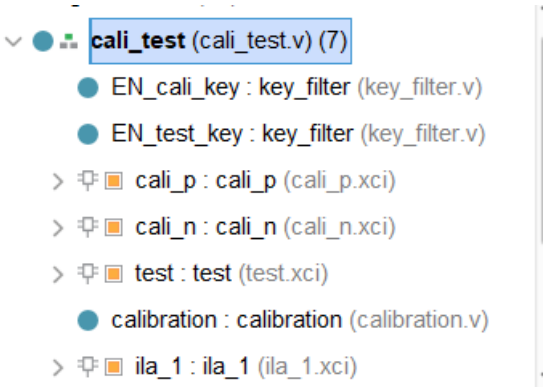
路径：F:\calibration\16bits\lower bits-based calibration\16bit_dither-ADC 模型
运行后，产生校准需要的数字码 dout1 和 dout2，和正常转换的数字码 dout。

dout	*1024x18 double*
dout1	*176x18 double*
dout2	*176x18 double*

图表 2-2 数字码提取

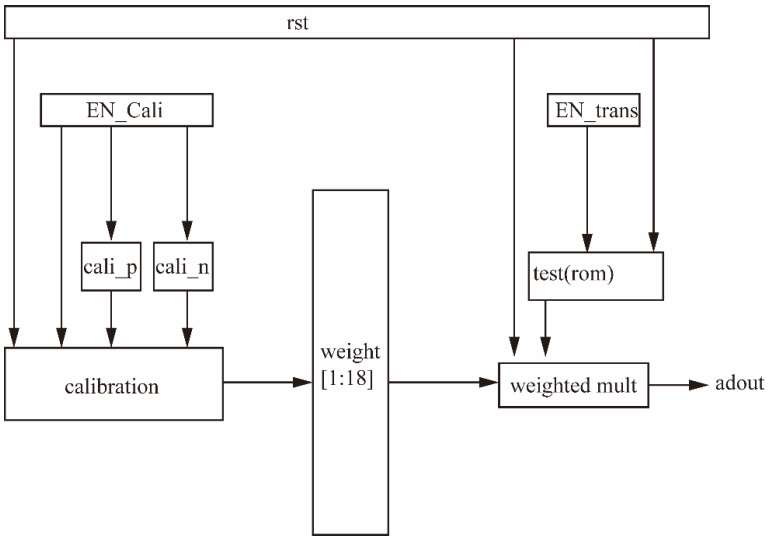
2.2. FPGA 工程与测试方式

2.2.1. 工程架构



图表 2-3 架构

大致的工程结构与基于扰动注入的数字前台校准模块类似，使用 rom 存储用于校准的数字码与用于测试的数字码。此处使用了 cali_p 和 cali_n 两个 rom 分别存储被校准位置 1 和置 0 时产生的数字码，简化了时序控制。test rom 中存储 ADC 对满摆幅的、 接近奈奎斯特频率的正弦波输入的量化点用于测试校准效果，此处设定用 1024 个点进行 fft。 Calibration 模块是校准算法的实现，传入被校准位置 1 和置 0 得到的数字码，输出 16 次校准取平均值得到的新权重，并将该权重运用于对下一位的校准中。同样使用 ILA 在线调试工具抓取最终的输出 adout；同样将 rst, EN_cali, EN_trans 三个控制信号与 FPGA 的用户按钮相连。



图表 2-4 架构图

具体介绍和测试结果见毕业论文 **4.1.2 低位校准高位的前台校准算法的 FPGA 硬件实现**

2.2.2. 测试流程

1. 使用 MATLAB 生成数字码，并导入 FPGA

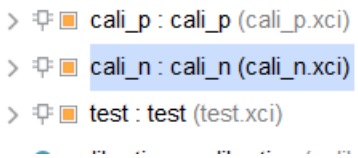
如前文所述，运行文件夹内的**\16bit_dither-ADC 模型**文件，分别得到校准的数字码和测试码。

用前文类似的方法，将数字码做成 coe 文件，共三个文件：

cali_n.coe	2025/4/2 20:00	COE 文件	4 KB
cali_p.coe	2025/4/2 20:00	COE 文件	4 KB
test.coe	2025/4/2 20:00	COE 文件	22 KB

图表 2-5 数字码 coe

与图 2-2 中的数字码分别对应，cali_p 写入 dout1，cali_n 写入 dout2，test 写入 dout，文件夹内有范例。**F:\calibration\16bits\lower bits-based calibration\coe**

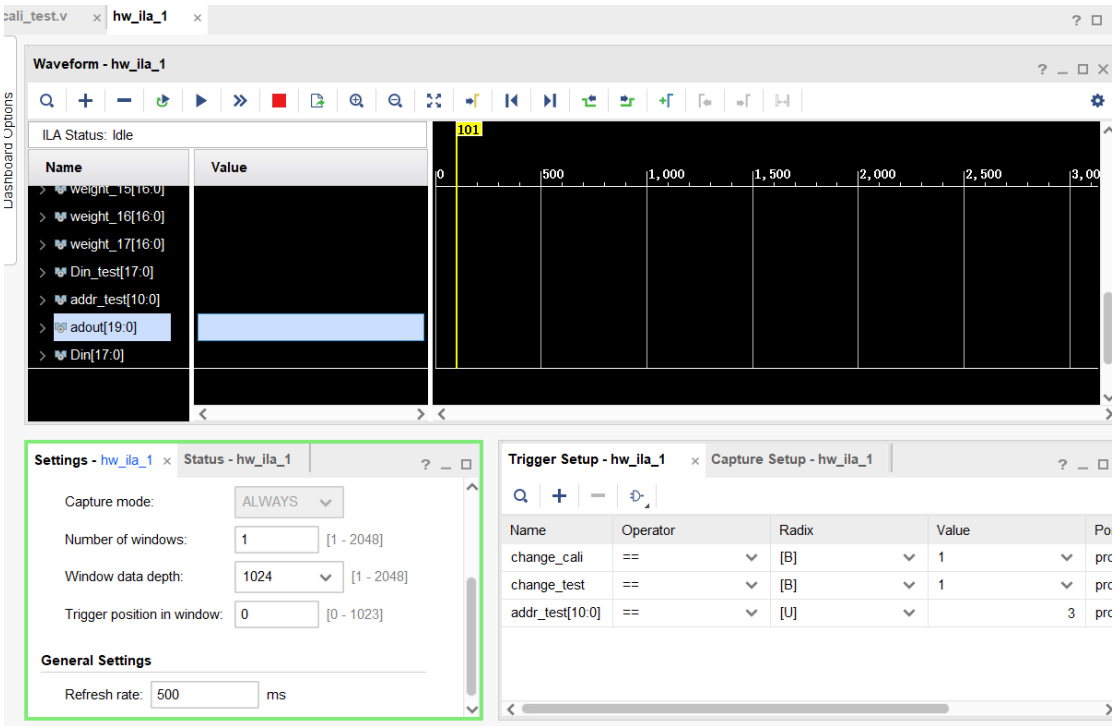


图表 2-6 数字码 rom

分别放入相同命名的 rom 内。

2. 进行测试

将比特流导入 FPGA 后，出现 ila 调试界面，按图设置好检测信号和触发信号，最终的数字码输出为 adout，共 20 位。：



图表 2-7 ila 设置

击运行后，开始检测触发信号，按照顺序分别按下 FPGA 的 PL LED2，PL LED3 按钮（分别是开始校准，开始重新编码的使能按钮，右 3 与右 2）

图表 2-8 按钮

得到 adout 的输出，右键波形设置为十进制输出后，在 tcl 输入命令

write_hw_ila_data -csv_file E:/XXX/XXX/FFT.csv [upload_hw_ila_data hw_ila_1]

此处设置了输出 csv 文件的路径与名字，得到的 csv 文件如下（FFT_djg）

I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
_N[17:(	weight_9[	weight_10	weight_11	weight_12	weight_13	weight_14	weight_15	weight_16	weight_17	Din_test[1	addr_test[	adout[19:(	Din[17:0]	
412	272	540	1074	1064	2150	4284	8596	17206	33324	10856	3	28843	15669	
412	272	540	1074	1064	2150	4284	8596	17206	33324	0c5d1	4	23439	10856	
412	272	540	1074	1064	2150	4284	8596	17206	33324	0871f	5	18334	0c5d1	
412	272	540	1074	1064	2150	4284	8596	17206	33324	5.50E+04	6	13668	0871f	
412	272	540	1074	1064	2150	4284	8596	17206	33324	02a0c	7	9569	5.50E+04	
412	272	540	1074	1064	2150	4284	8596	17206	33324	014a7	8	6154	02a0c	
412	272	540	1074	1064	2150	4284	8596	17206	33324	5.00E+05	9	3508	014a7	
412	272	540	1074	1064	2150	4284	8596	17206	33324	0060d	10	1712	5.00E+05	
412	272	540	1074	1064	2150	4284	8596	17206	33324	0151c	11	808	0060d	
412	272	540	1074	1064	2150	4284	8596	17206	33324	02a9b	12	825	0151c	
412	272	540	1074	1064	2150	4284	8596	17206	33324	056a6	13	1762	02a9b	
412	272	540	1074	1064	2150	4284	8596	17206	33324	8829	14	3592	056a6	
412	272	540	1074	1064	2150	4284	8596	17206	33324	0c6e6	15	6267	8829	
412	272	540	1074	1064	2150	4284	8596	17206	33324	109a4	16	9711	0c6e6	
412	272	540	1074	1064	2150	4284	8596	17206	33324	1.57E+08	17	13831	109a4	
412	272	540	1074	1064	2150	4284	8596	17206	33324	1a816	18	18515	1.57E+08	
412	272	540	1074	1064	2150	4284	8596	17206	33324	20971	19	23635	1a816	
412	272	540	1074	1064	2150	4284	8596	17206	33324	25aee	20	29048	20971	
412	272	540	1074	1064	2150	4284	8596	17206	33324	2aaeb	21	34609	25aee	
412	272	540	1074	1064	2150	4284	8596	17206	33324	2f91e	22	40165	2aaeb	
412	272	540	1074	1064	2150	4284	8596	17206	33324	33b6e	23	45560	2f91e	
412	272	540	1074	1064	2150	4284	8596	17206	33324	3.79E+10	24	50652	33b6e	
412	272	540	1074	1064	2150	4284	8596	17206	33324	3aae8	25	55298	3.79E+10	
412	272	540	1074	1064	2150	4284	8596	17206	33324	3d696	26	59373	3aae8	
412	272	540	1074	1064	2150	4284	8596	17206	33324	3eba5	27	62763	3d696	
412	272	540	1074	1064	2150	4284	8596	17206	33324	3fa2a	28	65377	3eba5	
412	272	540	1074	1064	2150	4284	8596	17206	33324	3f9e0	29	67142	3fa2a	

图表 2-9 csv 文件

注意，adout 数据位于 U 列。

打开 matlab 文件 F:\calibration\16bits\lower bits-based calibration\FFT

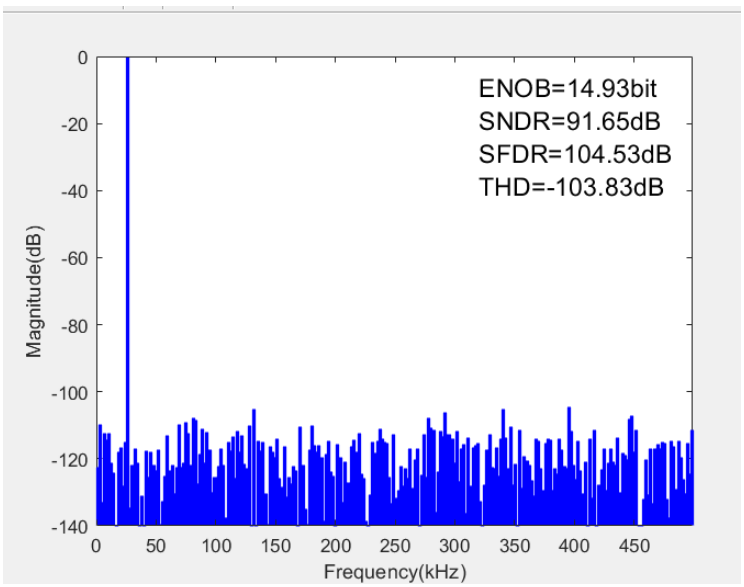
设置好 csv 文件的路径和数据的位置后，运行 test 主程序，即得到频谱

```
% clear all;
d_len=1024;
OSR=1;
filename = 'E:/2024_adc/DITHER/FFT_djg.csv';
% 检查文件是否存在
if ~isfile(filename)
    error('File not found: %s', filename);
end
% 读取 U 列的 2~1025 行数据
dataRange = 'U2:U1025';
% 使用 readmatrix 读取数据
M = readmatrix(filename, 'Range', dataRange);
MAT=M;

% 转置成行向量
adout = MAT';

[pow,SNR,SNDR,ENOB,SFDR,THD,HD] = calculate_dynamic_spec(adout);
%***************** Plot Output Power spectrum ****************************
figure(1);
base_value=-140;
xz = 0:d_len/2-1;
xz = xz / (d_len/2) *500 * OSR;
```

图表 2-10 FFT 设置



图表 2-11 FFT

测试结果见论文章节 4.1.3

3. 18bits，基于 dither 注入的 LMS 校准算法

3.1. 行为级模型

3.1.1. 原文件

路径：F:\calibration\18bits\dither-based LMS calibration\行为级模型\18bit_trans

程序的整体结构与框架与 16bits 的几乎没有区别，主程序的运行流程有所区别，参数设置有所区别，如电源电压，单位电容大小等；ADC 模块为 18bitsADC；LMS 校准模块为 cal_dither

1.非理想因素的设置如下：

```
del_Cu = Cu*0.03;
alfa =0.02; beta =0.02;
del_Compos = 3e-6;        % 定义比较器失调的均方根值
del_Compvn = 20*1e-6;     % |
pkt = 1;          %是否加入kT/C噪声
pkt_b = 0;      % 是否加入底板噪声，因为桥接电容寄生引起的？1加入，0不加入

%*****************以下是理想情况****************
% del_Cu = Cu*0;
% alfa =0; beta =0;
del_Compos = 0e-6;
del_Compvn = 0*1e-6;
pkt = 0;
%*************************************************
```

图表 3-1 非理想因素的设置

2.收敛因子的设置如下：

```
uk =2^(-9);%校准因子
u = [uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk,uk];

ud=u(7);
```

图表 3-2 收敛因子

需要注意，如前文介绍，在 16bits 中，若选取所有权重的收敛因子大小相同，uk 取 2^(-10)，这种取法的优点是硬件开销更加友好，在 verilog 中可以用移位代替乘法；

若选取不同权重的收敛因子大小不同，即 u＝weight*uk，则按照注释处设置，在林梦林的 ppt 中，这种取法会更快的收敛，但实际上两种取法在校准效果和收敛速度的区别不大，但前者的硬件开销更为友好。（参考文献中未看见取 u = weight*uk 这种做法）。

此处相比 16bits 的工程进行了优化，选取所有的收敛因子大小一致，没有进行 u= weight*uk 对其进行缩放。其优势在后面的 FPGA 工程的硬件开销十分显著。

3.主程序运行顺序：

首先进行校准前的转换，得到频谱图 figure1，若未开启非理想因素，ENOB 应为 17.99bits，若开启上图的非理想因素，ENOB 应在 10.6bits 左右波动。

```
%***************** SAR conversation ****************************
%校准前转换
L_real= log2((WL_dummy+sum(WLda) + W_Cc)/W_Cc);
Wda_real = Wda.*2^L_real;%高段因为低端的dummy加入，真实权重发生改变
Wdit_real=Wdit*2^L_real;
Wall(1:M+L+1) = [Wda_real WLda,1/2]; %所有位真实权重
weight =Wall(1:M+L+1);
dout=zeros(1,20);
dout_test=zeros(128,20);
for i = 1:d_len
    ctrl=0;
    Vip = 0.49*Vref*sin(i*delta_ph)+Vcm; % defining input signal 输入单频信号，幅度近似满量程，用sar adc对其进行转换，并将转换结果放到数组
    Vin = -0.49*Vref*sin(i*delta_ph)+Vcm;
    dout = adc_sar(Vip,Vin,Vref,Vcm,M,L,Cm_p,Cm_n,Cl_p,Cl_n,Cd1_p,Cd1_n,Cd2_p,Cd2_n,Ca_p,Ca_n,Cp1_p,Cp1_n,Cp2_p,Cp2_n,Cp3_p,Cp3_n, ...
        Comp_os,del_Compvn,del_ktc_p,del_ktc_n,Wda,WLda,W_Cc,WM_dummy,WL_dummy,del_ktc_b_p,del_ktc_b_n, Cdit_p,Cdit_n,ctrl);
    dout_test(i,:)=dout;
    dout=2*dout-1;
    adout(i) = dout * weight';
end

[pow,SNR,SNDR,ENOB,SFDR,THD,HD] = calculate_dynamic_spec(adout,OSR);
%***************** Plot Output Power spectrum ****************************
figure(1);
```

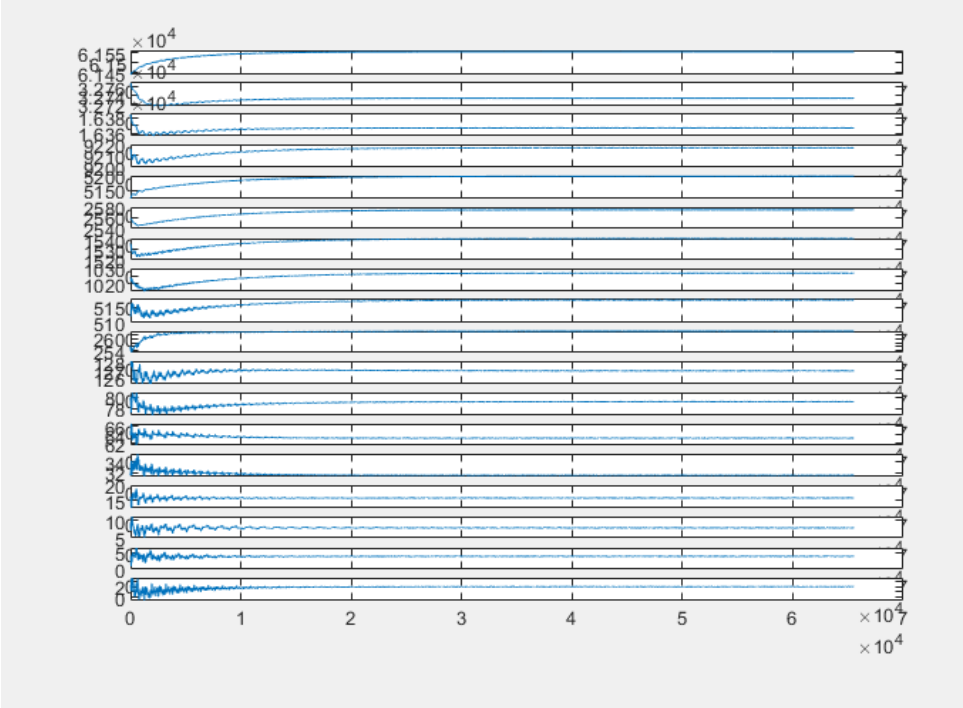
图表 3-3 校准前转换

随后进行低位校准高位的测试，调用了 calibration 程序，得到了对应的校准数字码，随后计算得到校准后的权重 weight_cali，输入信号进行测试，得到频谱 figure2。

```
%%%%%%%%%%%%%%%%%%%%%%%%%%低位校准高位%%%%%%%%%%%%%%%%%%%
```

Figure3 为权重随着迭代次数的变化曲线，有需要也可以进行查看:

```
% figure(3);
% subplot(size(weight_cal,2),1,1);    plot(weight_cal(:,1));
% subplot(size(weight_cal,2),1,2);    plot(weight_cal(:,2));
% subplot(size(weight_cal,2),1,3);    plot(weight_cal(:,3));
% subplot(size(weight_cal,2),1,4);    plot(weight_cal(:,4));
% subplot(size(weight_cal,2),1,5);    plot(weight_cal(:,5));
% subplot(size(weight_cal,2),1,6);    plot(weight_cal(:,6));
% subplot(size(weight_cal,2),1,7);    plot(weight_cal(:,7));
% subplot(size(weight_cal,2),1,8);    plot(weight_cal(:,8));
% subplot(size(weight_cal,2),1,9);    plot(weight_cal(:,9));
% subplot(size(weight_cal,2),1,10);    plot(weight_cal(:,10));
% subplot(size(weight_cal,2),1,11);    plot(weight_cal(:,11));
% subplot(size(weight_cal,2),1,12);    plot(weight_cal(:,12));
% subplot(size(weight_cal,2),1,13);    plot(weight_cal(:,13));
% subplot(size(weight_cal,2),1,14);    plot(weight_cal(:,14));
% subplot(size(weight_cal,2),1,15);    plot(weight_cal(:,15));
% subplot(size(weight_cal,2),1,16);    plot(weight_cal(:,16));
% subplot(size(weight_cal,2),1,17);    plot(weight_cal(:,17));
% subplot(size(weight_cal,2),1,18);    plot(weight_cal(:,18));
```



图表 3-6 权重与迭代次数的关系曲线

3.1.2. 其他程序与行为级测试结果

18bit_trans _10times：在 **18bit_trans** 基础上修改，连续进行 10 次仿真，用于比较低位校准高位和 dither 校准的效果，最后将 ENOB,SFDR,SNDR,THD 的数据导入 excel 制表。

18bit_trans _10times_learning curves：在上述的基础上提出了不同迭代次数的权重，用于绘制校准效果和迭代次数的关系曲线，最终选取迭代次数为 65536。

```
weight_1024 = weight_cal(1024,:);
weight_2048 = weight_cal(2048,:);
weight_4096 = weight_cal(4096,:);
weight_6144 = weight_cal(6144,:);
weight_8192 = weight_cal(8192,:);
weight_10238 = weight_cal(10238,:);
weight_12288 = weight_cal(12288,:);
weight_16384 = weight_cal(16384,:);
weight_24576 = weight_cal(24576,:);
weight_32768 = weight_cal(32768,:);
weight_65536 = weight_cal(65536,:);
weight_98304 = weight_cal(98304,:);
weight_end = weight_cal(end,:);
```

图表 3-7 提取不同迭代次数下的权重

测试结果：

行为级测试结果均记载在组会 ppt “wlx_5.22(6.3 更新)”，具体的原始数据在上述两个文件夹内的 xlxs 文件中。

10次仿真结果（不开启热噪声），Vref=5V

中山大学 集成电路学院

校准前	ENOB	SNDR	SFDR	THD	低位校准高位	ENOB	SNDR	SFDR	THD	LMS校准	ENOB	SNDR	SFDR	THD
平均值	10.37	64.19	70.56	-69.44		16.57	101.51	108.83	-109.35		17.84	109.13	116.32	-118.41
原始数据	10.06	62.35	68.22	-67.23		16.82	102.99	110.98	-109.88		18.04	110.39	119.11	-123.68
	10.62	65.67	70.27	-69.25		16.24	99.52	107.01	-105.55		18.29	111.88	120.30	-123.24
	10.42	64.46	70.75	-68.18		16.42	100.62	107.78	-108.21		17.44	106.74	112.59	-109.85
	10.74	66.44	73.34	-71.09		16.49	101.02	108.44	-111.63		17.65	108.00	113.50	-113.07
	10.59	65.48	72.39	-68.62		16.27	99.69	106.24	-107.60		17.90	109.51	117.19	-118.80
	9.67	59.96	65.46	-63.10		17.08	104.57	111.85	-110.74		17.80	108.89	118.51	-118.82
	10.57	65.40	72.92	-81.39		15.90	97.49	105.49	-104.07		17.74	108.54	116.58	-121.25
	10.22	63.29	68.81	-65.35		16.36	100.22	108.70	-107.51		17.78	108.82	114.75	-113.73
	11.01	68.05	75.93	-76.00		17.45	106.80	114.29	-113.38		17.90	109.49	116.05	-123.87
	9.80	60.76	67.51	-64.22		16.67	102.14	107.53	-114.92		17.82	109.05	114.63	-117.76

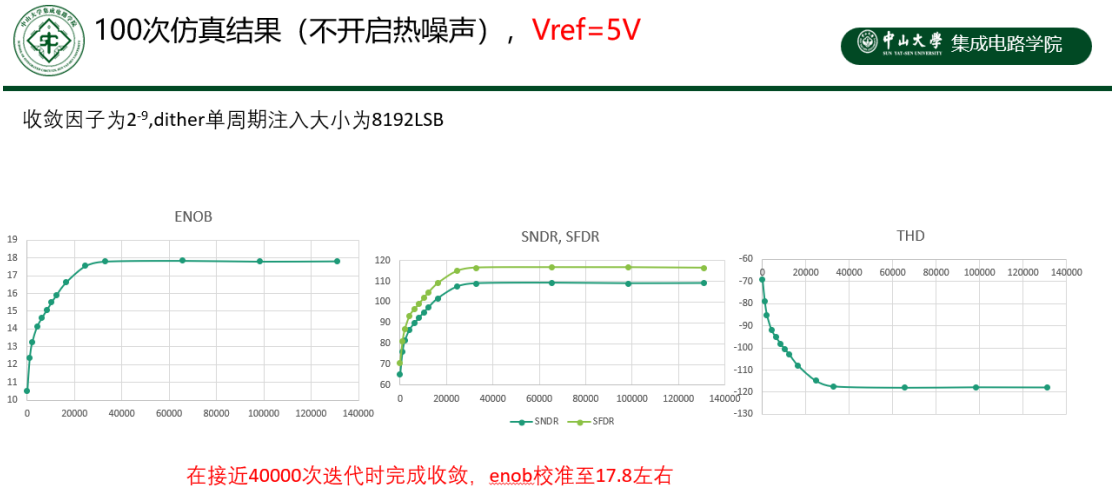
图表 3-8 与低位校准高位的比较（无热噪声）

10次仿真结果（开启热噪声），Vref=5V

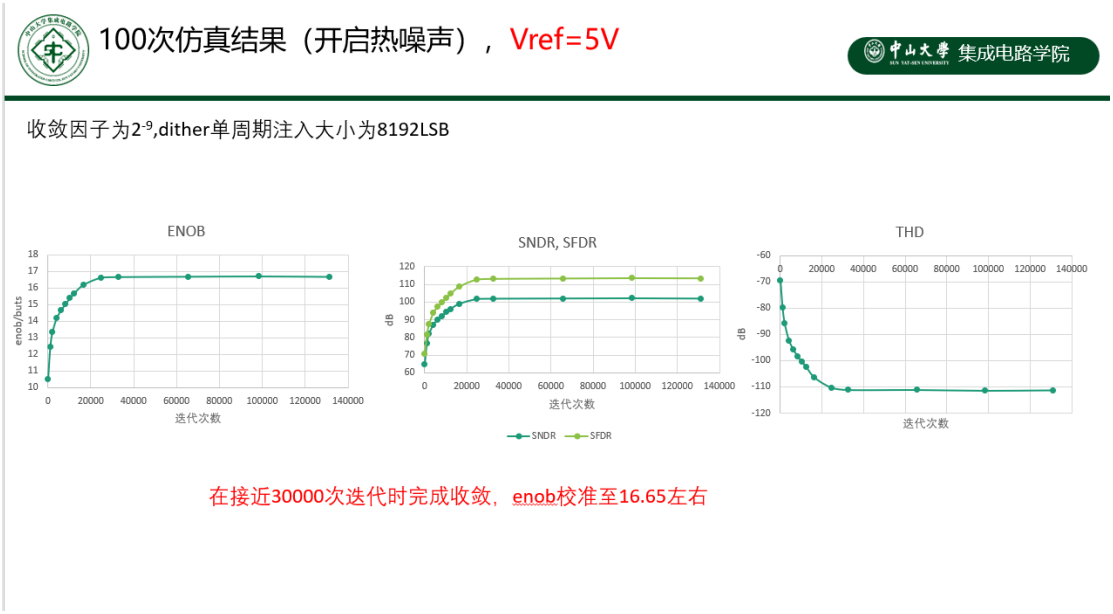
中山大学 集成电路学院

校准前	ENOB	SNDR	SFDR	THD	低位校准高位	ENOB	SNDR	SFDR	THD	LMS校准	ENOB	SNDR	SFDR	THD
平均值	10.50	64.98	70.61	-69.29		16.21	99.33	108.96	-107.43		16.69	102.24	114.41	-111.80
原始数据	9.57	59.38	62.62	-60.56		15.60	95.65	100.91	-98.99		16.57	101.52	113.33	-112.60
	11.01	68.05	75.15	-74.64		15.91	97.53	107.81	-104.09		16.58	101.58	113.96	-111.51
	11.55	71.29	77.93	-79.20		16.30	99.91	109.87	-107.51		16.82	103.03	115.68	-109.82
	9.96	61.71	67.20	-66.07		16.34	100.11	110.01	-108.53		16.71	102.36	115.61	-110.72
	10.24	63.43	71.34	-68.20		16.18	99.14	108.11	-106.72		16.77	102.69	115.29	-113.24
	10.32	63.88	68.66	-67.65		16.37	100.29	110.08	-108.79		16.63	101.89	112.03	-111.52
	10.78	66.67	72.51	-70.62		16.21	99.37	110.01	-112.67		16.70	102.32	114.62	-113.42
	10.84	67.04	74.57	-71.70		16.44	100.76	111.36	-108.01		16.68	102.18	114.38	-110.13
	10.73	66.37	69.55	-68.72		16.39	100.40	110.87	-109.67		16.72	102.39	114.69	-113.30
	10.00	61.97	66.58	-65.57		16.33	100.10	110.56	-109.27		16.73	102.50	114.52	-111.70

图表 3-9 与低位校准高位的比较（有热噪声）



图表 3-10 校准效果与迭代次数的关系曲线（无热噪声）



图表 3-11 图表 4 10 校准效果与迭代次数的关系曲线（有热噪声）

综上，取迭代次数为 65536（根据曲线，其实可以更少）。

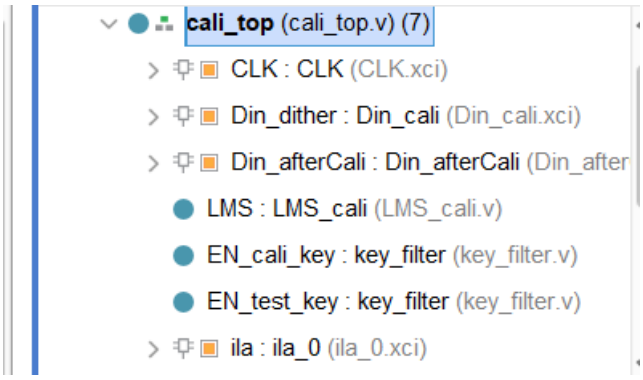
3.2. FPGA 工程与测试方式

工程为：
F:\calibration\18bits\dither-based LMS calibration\FPGA\18bits_ditFPGA

3.2.1. 工程架构：

整体的架构设置与 16bits 的完全一致，此处不进行赘述，使用了相同的 IP 核，数字码的导入方式完全一致，最终同样使用 ila 对 FPGA 的内部信号进行读

取。



图表 3-12 FPGA 工程

3.2.2. LMS 校准

本工程同样用 verilog 表示小数位，而小数位的设置则主要与收敛因子的大小直接关系。此工程的权重用 36 位 2 进制数表示，其构成为：

符号位（1 位）+整数位（17 位）+小数位（18 位），即 q18.18 的定点数格式

设计的小数位为 9 位，计算中会由于乘法进行位扩展，故取 18 位。但计算过程中也有部分 reg 的小数位只有 9 位，如 reg signed[26:0] VDAO_P，其格式为 q18.9。

```
77
78  //输出校准后的weight
79  always @(negedge CLK) begin
80  if (!rst) begin//同步置位
81  u<=27'b1;//用q18.9表示，收敛因子大小为2^(-9)
82   end
```

图表 3-13 收敛因子的设置

同样，收敛因子用 q18.9 的定点数格式表示，故此处 u 取二进制的 1，则实际表示 2^(9)。

```
167 else if (EN_inner) begin//LMS迭代
168     diff  = $signed(VDAO_P) - $signed(VDAO_N) - $signed(weight_dit);//计算err项，为q18.9格
169     delta =  {{9{diff[26]}}, diff};//err项进行符号位拓展，更为q18.18格式
170     for (j = 1; j < 20; j = j+1) begin
171      if(Din_P[j]>Din_N[j])
172       weight_fixp[j] <= $signed(weight_fixp[j]) -($signed({delta[26], delta}) << 1);
173       else if(Din_P[j]<Din_N[j])
174       weight_fixp[j] <= $signed(weight_fixp[j]) + ($signed({delta[26], delta}) << 1) ;
175       else weight_fixp[j] <= $signed(weight_fixp[j]);//与matlab的 weight_cal(i+1,j) = weig
176      if(j==19) begin
177        CNT<=CNT+1;
178       end
179      end
180      weight_dit<=weight_dit+($signed(delta)>>>8);//更新dither电容的“权重”
181 end
182 end
183
```

图表 3-14 LMS 迭代

与 16bits 的工程不同，所有权重的收敛因子大小均取相同的大小，因此可以

进行统一的移位来代替乘法的运行，因此极大的减小了硬件的开销。但是由于所有权重的收敛因子一致，每次迭代时低位权重的改变量相对最大(权重改变量/当前权重大小)，最低几位的在校准开始的几十次迭代内可能被更新至负数，因此，整个工程的计算均用有符号数表示，这一点与 16bits 的工程不同。

3.2.3. FPGA 工程测试

1. 数字码生成：

若用行为级 MATLAB 模型测试，可使用 matlab 文件：F:\calibration\18bits\dither-based LMS calibration\FPGA\18bit_trans – ADC

其在原文件 18bit_trans 修改，将校准阶段得到的数字码 dout_mem

和正常转换阶段的数字码 dout_test 提取到工作区，但运行时间会更长。

图表 3-15 数字码提取

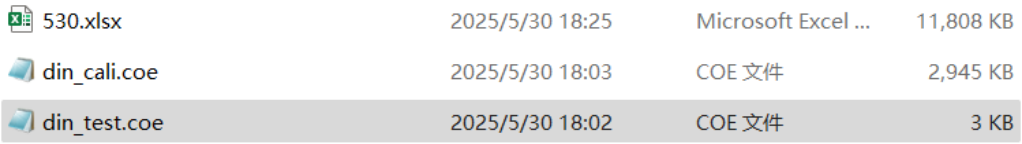
若使用电路前仿得到数字码，在仿真界面保存数字码输出后，在波形界面点击 measurements->analog to digital

2. 数字码导入

地址：F:\calibration\18bits\dither-based LMS calibration\FPGA\coe 5.30

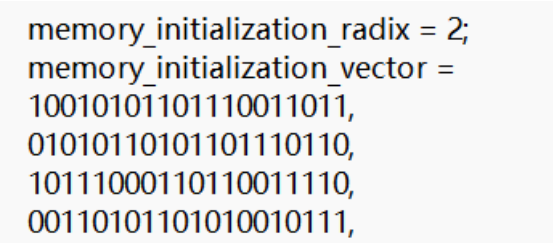
文件夹内的 5.30xlxs 为实例

将数字码复制到 excel 后拼接一个英文逗号，然后整列复制到 txt 文件中，按照语法输入提示词，注意最后一行以 Z “；”（英文符号）结束。然后将文件格式改为.coe，示例也在文件夹内。



530.xlsx	2025/5/30 18:25	Microsoft Excel ...	11,808 KB
din_cali.coe	2025/5/30 18:03	COE 文件	2,945 KB
din_test.coe	2025/5/30 18:02	COE 文件	3 KB

图表 3-16 文件范例



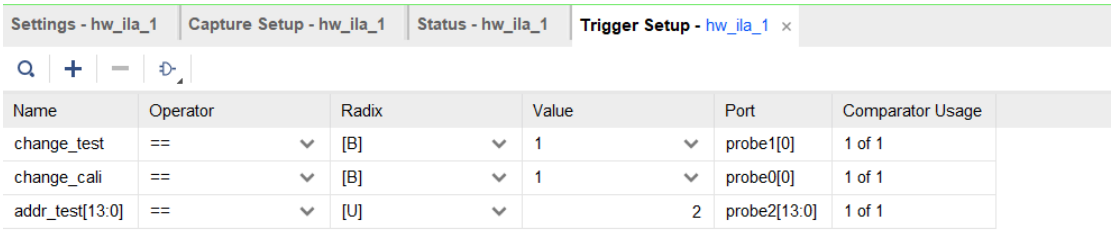
```
memory_initialization_radix = 2;
memory_initialization_vector =
10010101101110011011,
01010110101101110110,
10111000110110011110,
00110101101010010111,
```

图表 3-17 coe 文件格式（原文件在文件夹内）

5. 运行工程与结果输出

将比特流写入 FPGA 后，自动弹出 ila 调试界面，上方为监控 FPGA 内部信号的窗口，下方为触发信号的设置，按照图示设置：

图表 3-18 检测信号与位宽设置



图表 3-19 触发信号设置

点击运行后，开始检测触发信号，按照顺序分别按下 FPGA 的 PL LED2，PL LED3 按钮（分别是开始校准，开始重新编码的使能按钮，右 3 与右 2）

图表 3-20FPGA 按钮

得到 adout 的输出，右键波形设置为十进制输出后，在 tcl 输入命令

write_hw_ila_data -csv_file E:/2024_adc/DITHER/FFT.csv [upload_hw_ila_data hw_ila_1]

此处设置了输出 csv 文件的路径与名字，得到的 csv 文件如下（FFT617）：

A1 Sample in Buffer

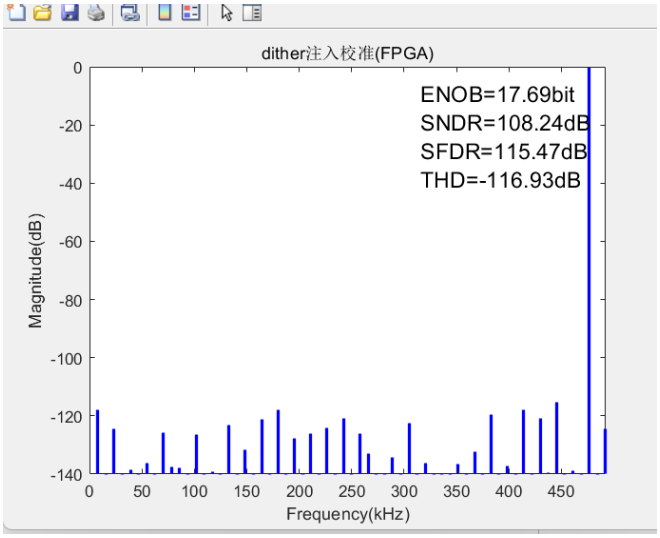
	A	B	C	D	E	F	G	H	I
1	Sample in	Sample in	TRIGGER	change_ca	change_te	addr_test[	Din_test[1	adin[35:0]	adout[20:0]
2	0	0	1	1	1	2	56b76	1.97E+10	1200108
3	1	1	0	1	1	3	b8d9e	1.23E+10	1200108
4	2	2	0	1	1	4	35a97	2.44E+10	750980
5	3	3	0	1	1	5	d7179	7.84E+09	1488711
6	4	4	0	1	1	6	19869	2.85E+10	478335
7	5	5	0	1	1	7	eeccd	4.18E+09	1739477
8	6	6	0	1	1	8	9136	3.16E+10	254877
9	7	7	0	1	1	9	fb562	1.64E+09	1930808
10	8	8	0	1	1	10	0196e	3.35E+10	99824
11	9	9	0	1	1	11	fea6a	4.35E+08	2046222
12	10	10	0	1	1	12	02f8d	3.4E+10	26543
13	11	11	0	1	1	13	f90c0	6.77E+08	2075779
14	12	12	0	1	1	14	0d343	3.3E+10	41343
15	13	13	0	1	1	15	e967b	2.34E+09	2016940
16	14	14	0	1	1	16	252af	3.07E+10	142951
17	15	15	0	1	1	17	ce58c	5.29E+09	1874770

FFT617

图表 3-21 csv 文件

6. 导入 MATLAB 进行 FFT

路径：F:\calibration\18bits\dither-based LMS calibration\FPGA\FFT - vdao 直接导入打开对应的 MATLAB 文件后，test 为主程序，正确设置好 csv 文件所在的路径后（如 filename＝'E:/2024_adc/18bits_may/FFT617.csv';，注意”/”的方向），即可读取数据进行 FFT。



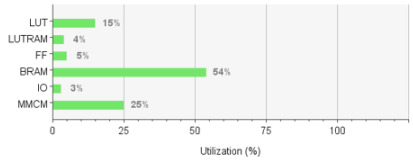
图表 3-22 读取 FPGA 内的数据进行 FFT

7. 测试结果：

18bits,LMS模块重写

中山大学 集成电路学院

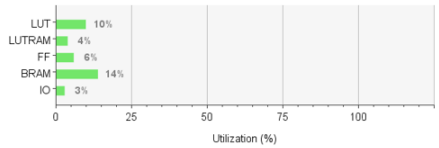
Resource	Utilization	Available	Utilization %
LUT	7896	53200	14.84
LUTRAM	645	17400	3.71
FF	5519	106400	5.19
BRAM	75.50	140	53.93
IO	4	125	3.20
MMCM	1	4	25.00



18bits LMS校准

- 所有权重的收敛因子完全一致，使用移位替代乘法(u*err)
- 为防止权重被减为负数，导致溢出，改用有符号数进行计算
- 输出21位数字码，在18位的基础上增加了3位小数位

Resource	Utilization	Available	Utilization %
LUT	5550	53200	10.43
LUTRAM	737	17400	4.24
FF	6131	106400	5.76
BRAM	19.50	140	13.93
IO	4	125	3.20

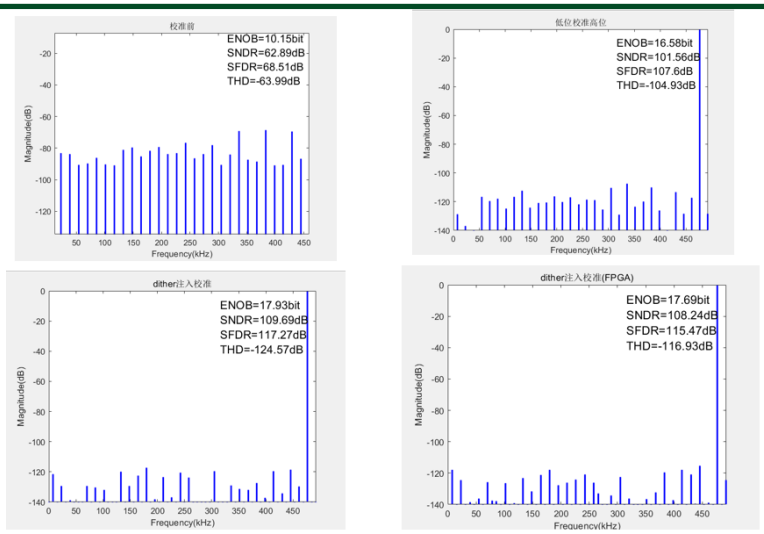


18bits 低位校准高位(3月的工程)

图表 3-23 硬件开销

18bits,LMS模块重写

中山大学 集成电路学院

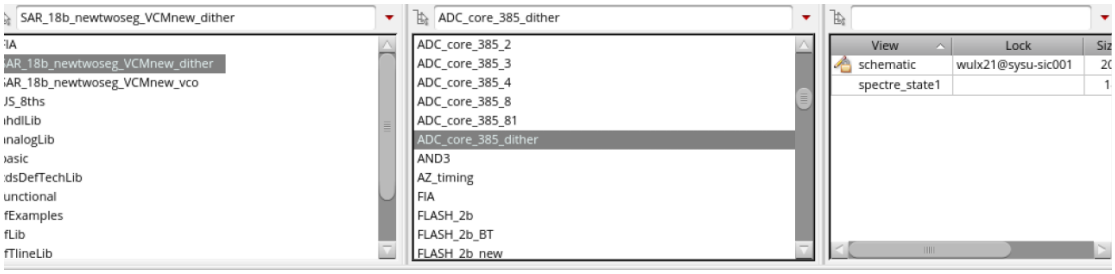


图表 3-24 校准效果

左上：打开非理想因素，3%的失配校准，2%的 alfabeta，不开启热噪声，校准前
右上：低位校准高位（全 matlab）
左下：dither 校准（全 matlab）
右下：dither 校准（matlab adc+FPGA 的 LMS 校准）

3.3. 电路修改

3.3.1. 电路位置

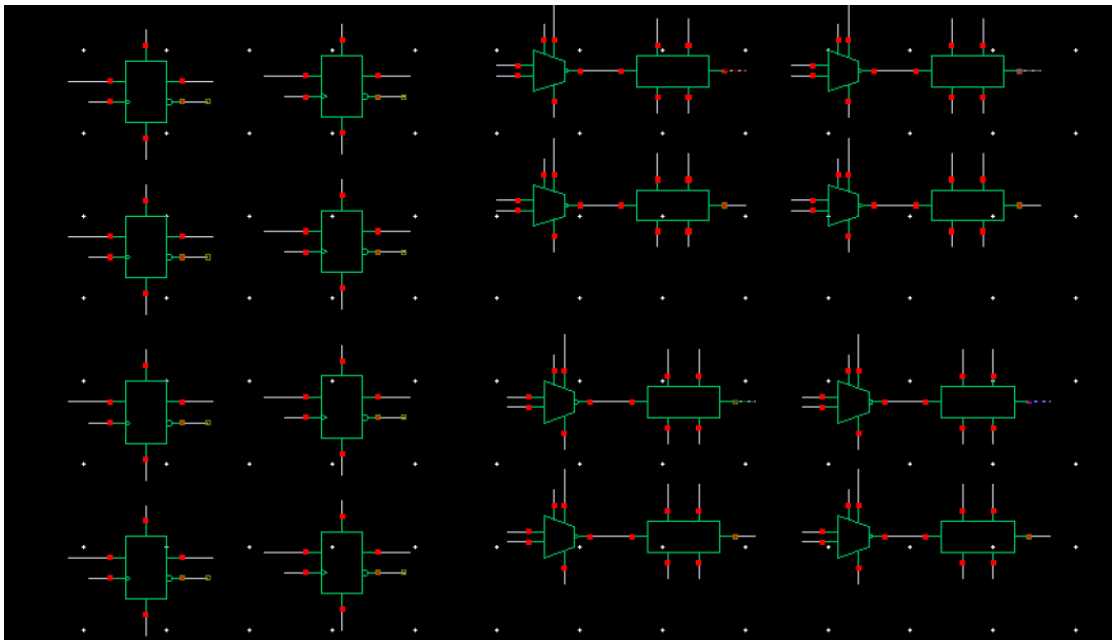


图表 3-25 电路位置

ADC_core_385_81 是实现了单次采样两次转换的电路(没加 dither)
ADC_core_385_dither 是加了 dither 的电路，还存在问题，ENOB 掉(见组会 ppt wlx6_19)

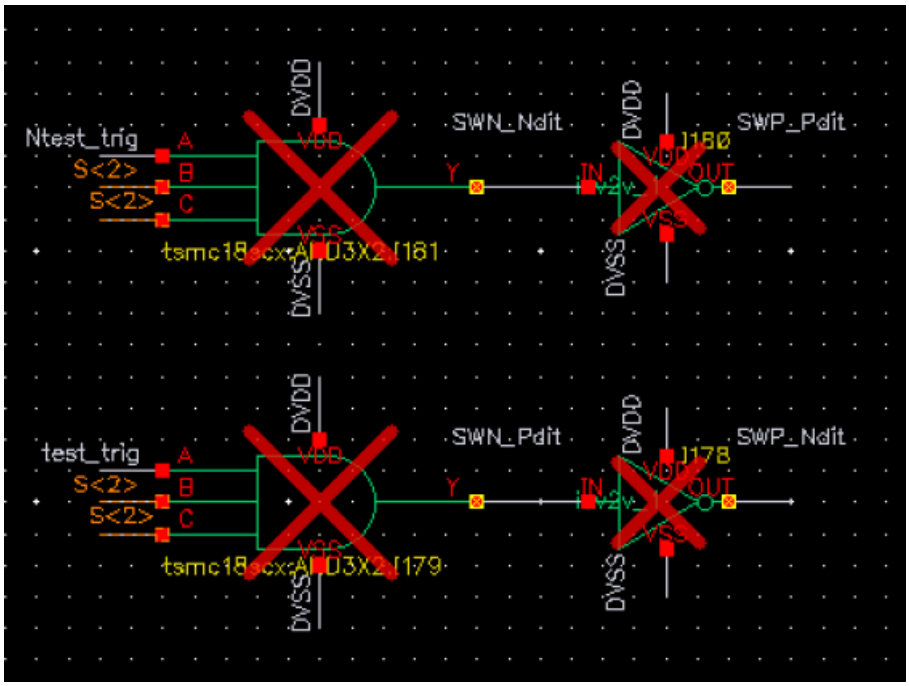
3.3.2. 电路修改内容

电路的修改内容与 16bits 类似，但更为初级。Flash 的输出锁存后再输出，在两次转换时保持一致。使得 MSB 电容在转换中不切换至 vcm，反正 DAC 输出超过 FIA 量程。



图表 3-26 flash 的锁存电路

Dither 电容的数字控制信号在逻辑上没有问题，但是仿真结果还存在问题。



图表 3-27 dither 电容的数字控制逻辑

3.3.3. 前仿测试的 matlab 文件

路径：F:\calibration\18bits\dither-based LMS calibration\行为级模型\18bit_trans -前仿测试

直接把前仿阶段产生的数字码导入（测试码与校准码），作用是代替 FPGA 工程，因为每次 FPGA 工程导入数字码后要重新写比特流，因此测试阶段用 matlab 会更有效率，测试无误后再将数字码导入 FPGA 进行验证。

测试方法：
打开 test，手动输入命令：
>> dout=0;
>> dout_cali=0;
右方工作区出现：

图表 3-28 数字码导入

点击矩阵，将数字码复制进去。得到：

图表 3-29 数字码导入

点击运行，即得到校准前频谱和校准后频谱。本工程在原文件 18bits_trans 上修改得到，简单而言，就是把原工程的对 adc 模块的调用注释掉，因此工程上无法通过仿真得到数字码，数字码改为由外部导入，其余部分没有改变。

4. 18bits，低位校准高位

4.1. 行为级模型

4.1.1. 原文件

路径：F:\calibration\18bits\lower bits-based calibration\MATLAB 行为级模型**18bit 针对 flash 的两分段**
没有进行任何修改，为原程序

4.1.2. 数字码产生文件

路径：F:\calibration\18bits\lower bits-based calibration\MATLAB 行为级模型\18bit 针对 flash 的两分段 - 数字码

在原程序的基础上将校准阶段产生的数字码提取出来，在主程序使用数字码计算校准后的电容。Dout 是用于 FFT 的 1024 的点，dout1 是校准阶段置 1 产生的数字码，dout2 是校准阶段置 0 产生的数字码。提取数字码便于后续 FPGA 工程的测试。

图表 4-1 数字码提取

4.2. FPGA 工程与测试方式

cali_FPGA	2025/6/19 22:02	文件夹	
coe	2025/4/2 20:20	文件夹	
FFT	2025/3/27 16:16	文件夹	
ila_dat1.csv	2025/6/19 22:04	Microsoft Excel ...	115 KB
mem_1.xlsx	2025/3/26 20:47	Microsoft Excel ...	320 KB
tcl.txt	2025/6/19 22:09	文本文档	1 KB

Cali_FPGA：FPGA 工程
Coe：FPGA rom 中的 coe 文件（示例）
FFT：matlab 代码，FPGA 的 ila 产生的纯二进制数字码导入进行 FFT
Ila_dat1:FPGA 导出的 csv 文件示例
Mem_1：处理数字码的示例文件
Tcl：tcl 命令示例

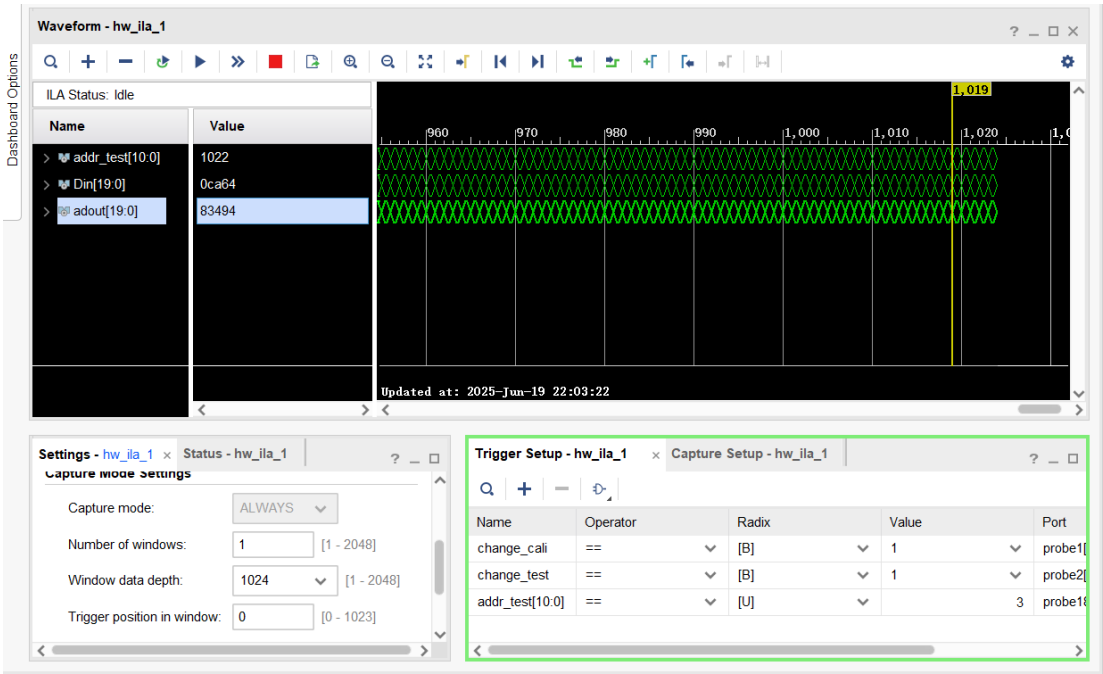
具体的测试流程与 16bits SAR ADC 低位校准高位完全一致，在此不赘述（实际上，16bits 的低校高 FPGA 工程是在 16bits 的低校高 FPGA 工程的基础上修改得到，故工程架构一样）。但有几点需要注意：

1. 16bits 的低位校准高位是校准 16 次取平均，18bits 的低位校准高位是 32 次取平均，因此校准阶段产生的数字码行数不同：

图表 4-2 16bits 数字码

图表 4-3 18bits 数字码

2. ila 触发信号设置：



图表 4-4 ila 触发信号

3. FPGA 的用户按钮逻辑完全一致，先按 PL2，后按 PL3。